

```
"""
fase2_bivariado.py
=====

FASE 2 - Pruebas bivariadas: poder predictivo individual frente al target.

Para cada variable sobreviviente de la fase 1 se calculan dos metricas
complementarias y se combinan en un score unico:

* **Information Value (IV)** sobre la discretizacion WOE. Mide la informacion
  acumulada bin a bin; capta relaciones no monotonas.
* **Gini** = 2*AUC-1. Mide la capacidad de ORDENAR la muestra por riesgo.

Tratamiento por tipo de variable (documentado por variable en la salida):

=====
Tipo                Tratamiento
=====
Numerica / booleana  Binning por cuantiles (igual frecuencia) con tamano
                     minimo de bin; nulos como bin propio. Gini calculado
                     sobre los valores crudos (`gini_bruto`) y sobre la
                     transformacion WOE (`gini_woe`).
Categorica           Categorías directas; las raras y el exceso de
                     cardinalidad se agrupan en `__OTROS__`. El Gini se
                     obtiene sobre la proyeccion WOE, porque una categoria
                     no tiene orden natural y el AUC crudo no esta definido.
Fecha               Se convierte a su representacion ordinal (dias desde
                     la epoca) y se trata como numerica.
=====

Metrica adicional propia del panel: **estabilidad temporal** (IV por periodo y
PSI). No se usa por defecto para excluir, pero se reporta, porque una variable
con IV alto concentrado en un solo periodo es una trampa en produccion.
"""

from __future__ import annotations

import numpy as np
import pandas as pd

from .config import ConfigPipeline
from .logging_utils import obtener_logger
from .metricas import (
    aplicar_woe,
    binear_categorica,
    binear_numerica,
    calcular_gini,
    clasificar_iv,
    estabilidad_temporal,
    gini_continuo,
    piso_ruido_gini,
    piso_ruido_iv,
    score_compuesto,
    tabla_woe,
)

LOGGER = obtener_logger("fase2")

def _preparar_serie(serie: pd.Series, tipo: str) -> tuple[pd.Series, str]:
    """Adapta la variable al calculo, devolviendo (serie, tipo_efectivo)."""
    if tipo == "FECHA":
        # Una fecha se ordena; se usa su ordinal para poder binearla.
        return pd.to_datetime(serie, errors="coerce").map(
            lambda d: d.toordinal() if pd.notna(d) else np.nan
        ), "NUMERICA"
    if tipo == "BOOLEANA":
        return pd.to_numeric(serie.astype("float"), errors="coerce"), "NUMERICA"
    return serie, tipo
```

```

def evaluar_variable(
    df: pd.DataFrame, col: str, tipo: str, y: pd.Series, cfg: ConfigPipeline,
    tipo_target: str,
) -> dict:
    """Calcula IV, Gini y estabilidad temporal para una variable."""
    resultado: dict = {
        "columna": col,
        "tipo_inferido": tipo,
        "iv": np.nan,
        "gini": np.nan,
        "gini_bruto": np.nan,
        "gini_woe": np.nan,
        "gini_signo": np.nan,
        "n_bins": np.nan,
        "metodo_binning": "",
        "metodo_gini": "",
        "clasificacion_iv": "NO_CALCULABLE",
        "n_observaciones_validas": 0,
        "observacion": "",
    }

    serie, tipo_efectivo = _preparar_serie(df[col], tipo)
    validas = serie.notna().sum()
    resultado["n_observaciones_validas"] = int(validas)

    # Sin datos suficientes no se inventa una metrica: se documenta el motivo.
    if validas < 30:
        resultado["observacion"] = (
            f"Solo {validas} observaciones no nulas (<30). IV y Gini no son estimables "
            "con fiabilidad; la variable se excluye por imposibilidad de evaluacion."
        )
        return resultado

    # --- 1. Discretizacion -----
    if tipo_efectivo in ("NUMERICA", "BOOLEANA"):
        bins, metodo = binear_numerica(serie, n_bins=cfg.n_bins, min_prop_bin=cfg.min_prop_bin)
    else:
        bins, metodo = binear_categorica(
            serie, max_categorias=cfg.max_categorias, min_prop_bin=cfg.min_prop_bin
        )
    resultado["metodo_binning"] = metodo
    resultado["n_bins"] = int(bins.nunique())

    if resultado["n_bins"] < 2:
        resultado["observacion"] = (
            "La discretizacion produjo un unico bin: la variable no separa la muestra. IV = 0."
        )
        resultado["iv"] = 0.0
        resultado["gini"] = 0.0
        resultado["clasificacion_iv"] = "SIN_PODER"
        return resultado

    # --- 2. Information Value -----
    if tipo_target == "BINARIO":
        y_iv = pd.to_numeric(y, errors="coerce")
    else:
        # Target continuo/multiclase: se binariza contra su mediana para poder
        # aplicar el marco WOE/IV. Es una simplificacion y se deja escrita.
        umbral = pd.to_numeric(y, errors="coerce").median()
        y_iv = (pd.to_numeric(y, errors="coerce") > umbral).astype(float)
        resultado["observacion"] += (
            f"Target no binario: para el IV se dicotomizo contra su mediana ({umbral:g}). "
        )

    tabla, iv = tabla_woe(bins, y_iv.values, correccion=cfg.correccion_woe)
    resultado["iv"] = float(iv)
    resultado["clasificacion_iv"] = clasificar_iv(iv)

    # --- 3. Gini -----
    mapa = dict(zip(tabla["bin"].astype(str), tabla["woe"].astype(float)))
    serie_woe = aplicar_woe(bins, mapa)

```

```

if tipo_target == "BINARIO":
    g_woe, g_woe_signo = calcular_gini(serie_woe, y)
    resultado["gini_woe"] = g_woe
    if tipo_efectivo in ("NUMERICA", "BOOLEANA"):
        g_bruto, g_signo = calcular_gini(serie, y)
        resultado["gini_bruto"] = g_bruto
        resultado["gini_signo"] = g_signo
        # Se toma el maximo: si la relacion es monotona ambos coinciden;
        # si es no monotona, el WOE la endereza y el bruto la subestima.
        # Quedarse con el bruto castigaria injustamente a la variable.
        resultado["gini"] = float(np.nanmax([g_bruto, g_woe]))
        resultado["metodo_gini"] = "max(AUC sobre valores crudos, AUC sobre WOE)"
    else:
        resultado["gini"] = g_woe
        resultado["gini_signo"] = g_woe_signo
        resultado["metodo_gini"] = "AUC sobre la proyeccion WOE (categorica sin orden natural)"
else:
    g, g_signo = gini_continuo(serie_woe, y)
    resultado["gini"] = g
    resultado["gini_woe"] = g
    resultado["gini_signo"] = g_signo
    resultado["metodo_gini"] = "|Spearman| WOE vs target (aproximacion de Somers para target continuo)"

# --- 4. Estabilidad temporal (panel) -----
try:
    est = estabilidad_temporal(bins, y_iv, df[cfg.columna_tiempo], correccion=cfg.correccion_woe)
    resultado.update(est)
except Exception as exc: # noqa: BLE001 - metrica complementaria, no bloquea
    LOGGER.debug("Estabilidad temporal no calculable para '%s': %s", col, exc)

return resultado

def ejecutar(
    df: pd.DataFrame,
    cfg: ConfigPipeline,
    tipos: dict[str, str],
    sobrevivientes: list[str],
    tipo_target: str,
) -> tuple[pd.DataFrame, dict[str, pd.DataFrame]]:
    """Ejecuta la fase bivariada.

    Returns
    -----
    (tabla_bivariada, tablas_woe)
        La segunda salida guarda la tabla WOE detallada de cada variable, que
        se anexa al Excel como evidencia del calculo del IV.
    """
    LOGGER.info("=" * 78)
    LOGGER.info("FASE 2 - PRUEBAS BIVARIADAS (IV / GINI)")
    LOGGER.info(
        "Variables a evaluar=%d | bins=%d | min_prop_bin=%.2f | pesos gini/iv=%.2f/%.2f (%s)",
        len(sobrevivientes), cfg.n_bins, cfg.min_prop_bin, cfg.peso_gini, cfg.peso_iv,
        cfg.metodo_normalizacion,
    )
    LOGGER.info("=" * 78)

    if not sobrevivientes:
        LOGGER.warning("No hay variables que evaluar en la fase bivariada.")
        return pd.DataFrame(), {}

    # Solo filas con target informado: una observacion sin target no puede
    # contribuir al calculo de poder predictivo.
    y_completo = pd.to_numeric(df[cfg.columna_target], errors="coerce")
    mascara = y_completo.notna()
    if (~mascara).any():
        LOGGER.info("Se descartan %d filas con target nulo para las metricas bivariadas.",
                    int((~mascara).sum()))
    dfx = df.loc[mascara].reset_index(drop=True)
    y = y_completo.loc[mascara].reset_index(drop=True)

```

```

filas, tablas_woe = [], {}
for i, col in enumerate(sobrevivientes, start=1):
    try:
        res = evaluar_variable(dfx, col, tipos.get(col, "CATEGORICA"), y, cfg, tipo_target)
        filas.append(res)

        # Se reconstruye la tabla WOE para la evidencia (solo si es util).
        if res["n_bins"] and res["n_bins"] >= 2:
            serie, tef = _preparar_serie(dfx[col], tipos.get(col, "CATEGORICA"))
            if tef in ("NUMERICA", "BOOLEANA"):
                bins, _ = binear_numerica(serie, cfg.n_bins, cfg.min_prop_bin)
            else:
                bins, _ = binear_categorica(serie, cfg.max_categorias, cfg.min_prop_bin)
            t, _iv = tabla_woe(bins, y.values, cfg.correccion_woe)
            t.insert(0, "columna", col)
            tablas_woe[col] = t
    except Exception as exc: # noqa: BLE001
        LOGGER.error("Error evaluando '%s': %s. Se marca como no evaluable.", col, exc)
        filas.append(
            {"columna": col, "tipo_inferido": tipos.get(col, "?"), "iv": np.nan,
             "gini": np.nan, "observacion": f"Error de calculo: {exc}"}
        )

    if i % 25 == 0 or i == len(sobrevivientes):
        LOGGER.info("    ... %d/%d variables evaluadas", i, len(sobrevivientes))

biv = pd.DataFrame(filas)

# -----
# Score compuesto
# -----
scores = score_compuesto(
    biv["gini"], biv["iv"], cfg.peso_gini, cfg.peso_iv, cfg.metodo_normalizacion
)
biv = pd.concat([biv, scores], axis=1)
biv["peso_gini"] = cfg.peso_gini
biv["peso_iv"] = cfg.peso_iv

# -----
# Reglas de exclusion
# -----
biv["flg_no_evaluable"] = (biv["iv"].isna() & biv["gini"].isna()).astype(int)

# Regla principal: se excluye solo si falla en AMBAS metricas. Usar "o"
# eliminaria variables que una de las dos metricas capta bien (tipico de
# relaciones no monotonas, donde el Gini crudo se hunde y el IV no).
# --- Piso de ruido: umbral adaptativo por variable -----
# El umbral efectivo es el MAYOR entre el fijo (parametro del usuario) y el
# piso estadistico, que depende del tamano muestral y del numero de bins de
# ESA variable. Asi una variable con 20 bins no compite contra el mismo
# baremo que una con 3.
n_ev = int(pd.to_numeric(y, errors="coerce").sum())
n_no = int(len(y) - n_ev)
n_contrastes = len(biv) if cfg.bonferroni_ruido else 1

if cfg.usar_piso_ruido and n_ev > 0 and n_no > 0:
    biv["piso_ruido_iv"] = biv["n_bins"].apply(
        lambda k: piso_ruido_iv(n_ev, n_no, int(k), cfg.alpha_ruido, n_contrastes)
        if pd.notna(k) and k >= 2 else np.nan
    )
    piso_g = piso_ruido_gini(n_ev, n_no, cfg.alpha_ruido, n_contrastes)
    biv["piso_ruido_gini"] = piso_g
    biv["umbral_iv_efectivo"] = np.maximum(
        cfg.umbral_iv_minimo, biv["piso_ruido_iv"].fillna(cfg.umbral_iv_minimo)
    )
    biv["umbral_gini_efectivo"] = max(cfg.umbral_gini_minimo, piso_g if np.isfinite(piso_g) else 0)
    LOGGER.info(
        "Piso de ruido activo (alpha=%3g): con %d eventos y %d no eventos, "
        "una variable aleatoria de 10 bins alcanzaria IV<=%4f y Gini<=%4f solo por azar.",
        cfg.alpha_ruido, f", Bonferroni n={n_contrastes}" if cfg.bonferroni_ruido else "",

```

```

        n_ev, n_no, piso_ruido_iv(n_ev, n_no, 10, cfg.alpha_ruido, n_contrastes), piso_g,
    )
else:
    biv["piso_ruido_iv"] = np.nan
    biv["piso_ruido_gini"] = np.nan
    biv["umbral_iv_efectivo"] = cfg.umbral_iv_minimo
    biv["umbral_gini_efectivo"] = cfg.umbral_gini_minimo

# Que Gini se contrasta contra el umbral: el CRUDO, no el de WOE.
# `gini_woe` se calcula sobre un mapeo estimado con los mismos datos, de modo
# que consume k-1 grados de libertad y esta sesgado al alza: una variable
# aleatoria de 10 bins alcanza un gini_woe apreciable solo por sobreajuste.
# Compararlo contra un piso derivado del AUC no ajustado seria comparar
# peras con manzanas y dejaria pasar ruido puro.
#
# - Numericas -> se usa `gini_bruto`, que no ajusta ningun parametro.
# - Categoricas-> no existe un Gini no ajustado (no hay orden natural), asi
# que no aportan evidencia independiente del IV y la decision recae
# integramente en el IV, cuyo piso SI incorpora los grados de libertad.
if cfg.usar_piso_ruido:
    gini_contraste = biv["gini_bruto"]
    biv["gini_contrastado"] = gini_contraste
    biv["fuente_gini_umbral"] = np.where(
        gini_contraste.notna(), "gini_bruto (sin ajuste)",
        "no aplica: categorica sin Gini no ajustado; decide el IV",
    )
    # NaN -> se considera "bajo": no puede rescatar a la variable.
    biv["flg_gini_bajo"] = (
        gini_contraste.fillna(-np.inf) < biv["umbral_gini_efectivo"]
    ).astype(int)
else:
    biv["gini_contrastado"] = biv["gini"]
    biv["fuente_gini_umbral"] = "gini (max de bruto y WOE)"
    biv["flg_gini_bajo"] = (biv["gini"].fillna(0) < biv["umbral_gini_efectivo"]).astype(int)

biv["flg_iv_bajo"] = (biv["iv"].fillna(0) < biv["umbral_iv_efectivo"]).astype(int)
biv["flg_supera_ruido"] = ((biv["flg_iv_bajo"] == 0) | (biv["flg_gini_bajo"] == 0)).astype(int)
biv["flg_sospecha_fuga"] = (biv["iv"].fillna(0) > cfg.umbral_iv_sospechoso).astype(int)
biv["flg_inestable_temporal"] = (
    biv.get("psi_max", pd.Series(np.nan, index=biv.index)).fillna(0) > cfg.umbral_psi
).astype(int)

motivos: list[str] = []
excluir: list[int] = []
for _, f in biv.iterrows():
    razones: list[str] = []
    if f["flg_no_evaluable"]:
        razones.append(f"no evaluable ({f.get('observacion', '')})")
    if f["flg_iv_bajo"] and f["flg_gini_bajo"]:
        g_txt = (f"Gini={f['gini_contrastado']:.4f} < {f['umbral_gini_efectivo']:.4f}"
                 if pd.notna(f.get("gini_contrastado"))
                 else "sin Gini no ajustado disponible (categorica)")
        razones.append(
            f"IV={f['iv']:.4f} < {f['umbral_iv_efectivo']:.4f} y {g_txt} "
            "(sin poder predictivo en ninguna metrica"
            + (f"; umbrales elevados al piso de ruido para {int(f['n_bins'])} bins "
              f"y {n_ev} eventos)" if cfg.usar_piso_ruido
              and f["umbral_iv_efectivo"] > cfg.umbral_iv_minimo else ")")
    )
    if cfg.umbral_score_minimo > 0 and pd.notna(f["score_compuesto"]) \
        and f["score_compuesto"] < cfg.umbral_score_minimo:
        razones.append(f"score compuesto={f['score_compuesto']:.4f} < {cfg.umbral_score_minimo}")
    if cfg.excluir_sospecha_fuga and f["flg_sospecha_fuga"]:
        razones.append(f"IV={f['iv']:.4f} > {cfg.umbral_iv_sospechoso} (posible fuga de informacion)")
    if cfg.excluir_por_inestabilidad and f["flg_inestable_temporal"]:
        razones.append(f"PSI maximo={f.get('psi_max', np.nan):.4f} > {cfg.umbral_psi} (inestable en el panel)")
    excluir.append(int(bool(razones)))
    motivos.append("; ".join(razones))

biv["flg_exclusion"] = excluir
biv["motivo_exclusion_bivariada"] = motivos

```

```

# Top-N opcional: se aplica DESPUES de las reglas absolutas.
if cfg.top_n_bivariado > 0:
    vivas = biv[biv["flg_exclusion"] == 0].sort_values("score_compuesto", ascending=False)
    fuera = vivas.iloc[cfg.top_n_bivariado:]["columna"]
    mask = biv["columna"].isin(fuera)
    biv.loc[mask, "flg_exclusion"] = 1
    biv.loc[mask, "motivo_exclusion_bivariada"] += (
        f"; fuera del top-{cfg.top_n_bivariado} por score compuesto"
    )

biv["flg_seleccionada_bivariada"] = 1 - biv["flg_exclusion"]

# Advertencias que NO excluyen pero deben verse.
avisos = []
for _, f in biv.iterrows():
    a = []
    if f["flg_sospecha_fuga"] and not cfg.excluir_sospecha_fuga:
        a.append(f"IV={f['iv']:.3f} anormalmente alto: revise fuga de informacion")
    if f["flg_inestable_temporal"] and not cfg.excluir_por_inestabilidad:
        a.append(f"PSI={f.get('psi_max', np.nan):.3f}: distribucion inestable entre periodos")
    cvp = f.get("iv_cv_periodo", np.nan)
    if pd.notna(cvp) and cvp > 0.75:
        a.append(f"IV muy variable entre periodos (CV={cvp:.2f})")
    avisos.append("; ".join(a))
biv["advertencias"] = avisos

biv = biv.sort_values("score_compuesto", ascending=False, na_position="last").reset_index(drop=True)
biv["ranking_score"] = np.arange(1, len(biv) + 1)

n_out = int(biv["flg_exclusion"].sum())
LOGGER.info("Variables evaluadas          : %d", len(biv))
LOGGER.info("Excluidas en la fase 2        : %d", n_out)
LOGGER.info("Sobrevivientes de la fase 2    : %d", len(biv) - n_out)
if len(biv):
    top = biv.head(5)[["columna", "iv", "gini", "score_compuesto"]]
    LOGGER.info("Top 5 por score compuesto:\n%s", top.to_string(index=False))
if int(biv["flg_sospecha_fuga"].sum()):
    LOGGER.warning(
        "%d variable(s) con IV > %.2f: posible fuga de informacion. Revise su construccion.",
        int(biv["flg_sospecha_fuga"].sum()), cfg.umbral_iv_sospechoso,
    )

return biv, tablas_woe

def obtener_sobrevivientes(biv: pd.DataFrame) -> list[str]:
    """Variables que pasan a la fase multivariada."""
    if biv.empty():
        return []
    return biv.loc[biv["flg_exclusion"] == 0, "columna"].tolist()

```